ZMS LAB · PLATFORM ENGINEERING

Ansible for the ZMS Lab

A Practical Training Course & Codebase Reference

Configuration management for a dynamically discovered AWS estate — Ubuntu, Amazon Linux, and Windows Server — using the push model.

Repository	ZMS-AWS-Ansible-2 (`control-repo/`)
Control model	Push — SSH (Linux), WinRM/HTTPS (Windows)
Discovery	EC2 dynamic inventory (`amazon.aws.aws_ec2`)
Secrets	AWS Secrets Manager — nothing sensitive in Git
Audience	IT/ops engineers new to Ansible

Contents

PART I

Ansible Fundamentals

No prior Ansible experience assumed. If you're comfortable with Linux, AWS, and a terminal, this part gets you fluent in the concepts before we open a single file in the repo.

1.1 What Ansible is, and the problem it solves

Ansible is an open-source **configuration-management and automation** engine. Instead of writing step-by-step scripts ("run this, then that"), you describe the *desired state* of your machines in readable YAML — "this package is present, this service is running, this file contains this text" — and Ansible makes reality match. Three properties make that powerful:

- **Agentless.** Nothing permanent runs on the managed machines. Ansible connects on demand over protocols they already speak — **SSH** for Linux, **WinRM** for Windows — does its work, and disconnects. A Linux host needs only an SSH server and Python; a Windows host needs a WinRM listener and PowerShell.
- **Idempotent.** Running the same playbook twice is safe. A well-written task checks current state first and only changes what differs. The first run installs a package (`changed`); the second sees it present and reports `ok` . This is what turns Ansible into a *drift-correction* tool, not a one-shot installer.
- **Declarative & readable.** The YAML doubles as documentation. A teammate can read a playbook and know what the estate looks like without running anything.

> **Why it matters here**
>
> Our estate is rebuilt from Terraform and changes shape often (hosts come and go across availability zones). A declarative, idempotent tool that *discovers* hosts at run time — rather than a script with hard-coded IPs — is exactly the right fit.

1.2 Push vs. pull — the model this repo uses

Ansible can run two ways. In **pull** mode each host periodically fetches and applies its own config. In **push** mode a central **control node** connects outward and applies config to the hosts it discovers. This repository is built entirely around **push**: one EC2 control node inside the VPC holds this Git repo and pushes configuration to every instance it finds.

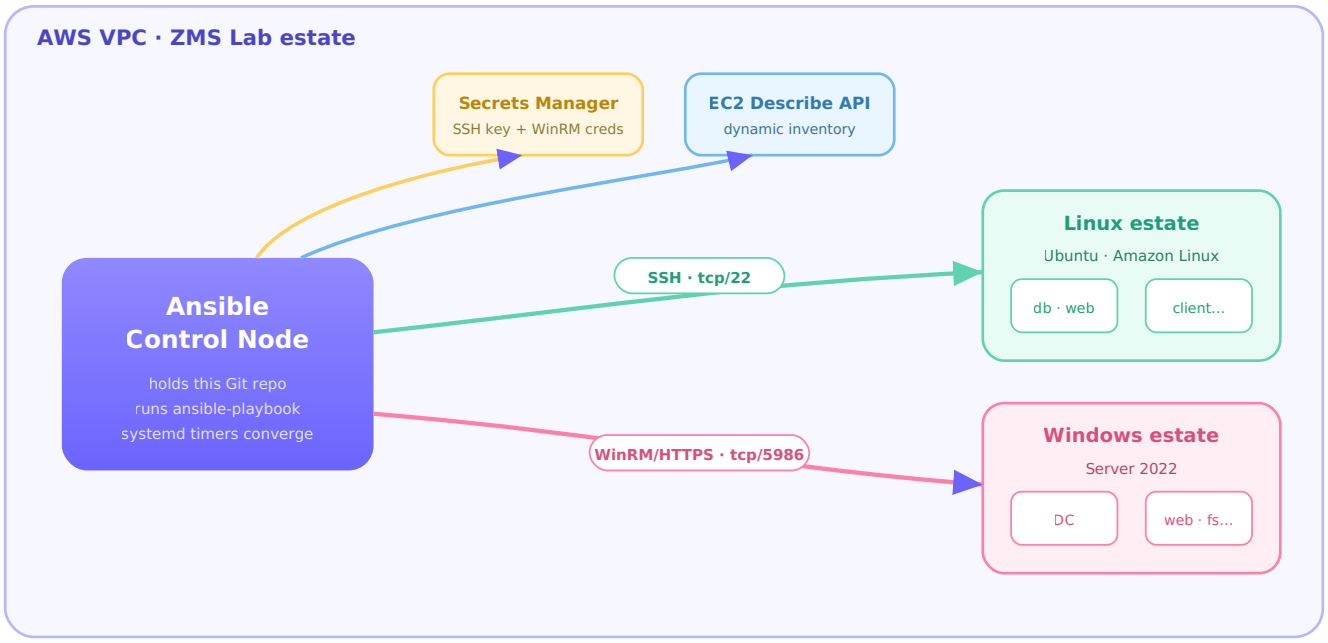

Figure 1 — The push model. One control node reads credentials and the host list from AWS, then pushes configuration outward over SSH and WinRM. No agent runs on the managed hosts.

1.3 The core vocabulary

Meet these terms once; the rest of the guide leans on them.

Term	What it means (and where it lives here)
Control node	The machine that runs `ansible-playbook`. Here, an EC2 instance in the VPC with IAM rights to describe EC2 and read one secret.
Managed node	Any host Ansible configures — our Ubuntu, Amazon Linux, and Windows instances.
Inventory	The list of managed nodes and their groups. Ours is *dynamic*: `inventory/aws_ec2.yml` queries EC2 at run time.
Module	A single-purpose unit of work shipped to a host, e.g. `ansible.builtin.apt`, `ansible.windows.win_service`. Idempotent by design.
Collection	A versioned bundle of modules/plugins from Ansible Galaxy, e.g. `amazon.aws`, `ansible.windows`. Ours are pinned in `requirements.yml`.
Task	One call to one module, with a human name. The unit of change.
Play	A mapping of a group of hosts to an ordered list of tasks/roles.
Playbook	A YAML file containing one or more plays, e.g. `site.yml`.
Role	A reusable, structured bundle of tasks/vars/handlers, e.g. our `baseline` role.
group_vars	Variables applied to every host in a group, e.g. connection settings in `group_vars/os_windows.yml`.
Facts	Data Ansible gathers about a host at connect time (OS family, IPs…), used in `when:` conditions.
Handler	A task that runs only when *notified* by a changed task — typically "restart service".

become	Privilege escalation (`sudo` on Linux). Enabled globally in `ansible.cfg` .
Idempotency	The property that re-running changes nothing already correct. Your north star when writing tasks.

1.4 What happens when you run a playbook

When you type `ansible-playbook site.yml` , Ansible walks a predictable pipeline. Understanding it makes error messages obvious.

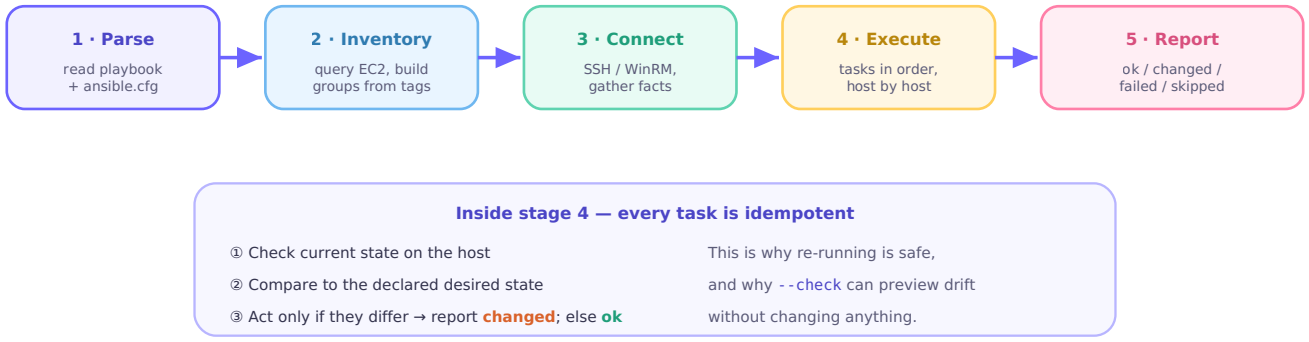

Figure 2 — The run pipeline. Parse → build inventory → connect & gather facts → execute tasks in order → report a per-host tally.

Read the tally

Every run ends with a `PLAY RECAP` line per host: `ok=` tasks that matched, `changed=` tasks that acted, `failed=` , `unreachable=` (couldn't connect), `skipped=` . A healthy converge on a steady estate shows `changed=0 failed=0 unreachable=0` .

PART II

How This Repository Is Built

Where Terraform ends and Ansible begins, what every file does, and how the control node keeps itself and the estate converged without anyone logging in.

2.1 The big picture: Terraform builds, Ansible configures

Two tools, one clean handoff. **Terraform** provisions infrastructure — the VPC, the EC2 instances, their tags, the one Secrets Manager secret, and the control node itself. **Ansible** (this repo) then *configures* those instances. The only things that cross the boundary are: the instance **tags** (which become inventory groups), the **secret name** (injected into the control node), and the **SSH key** (stored in the secret).

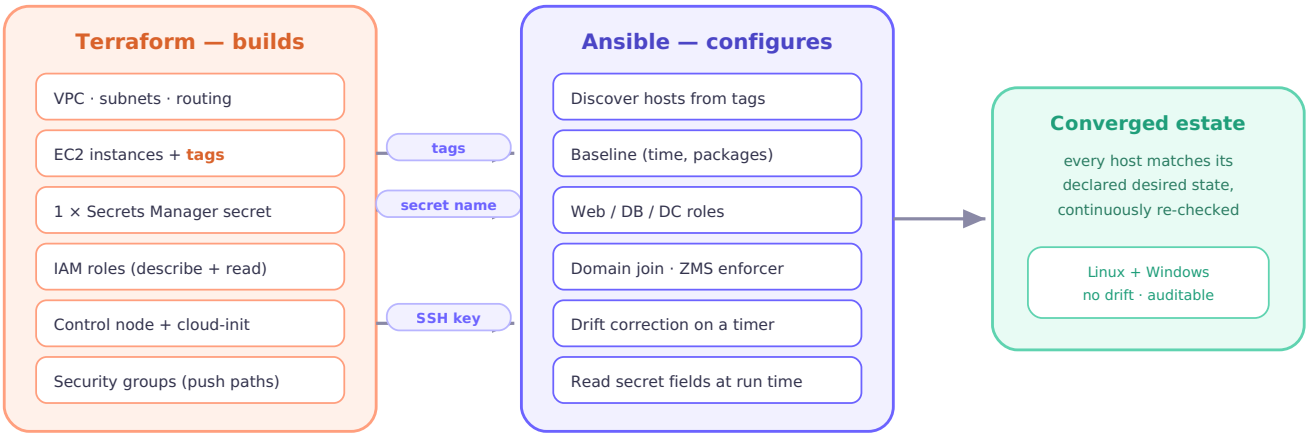

Figure 3 — Terraform provisions and hands off three things (tags, secret name, SSH key); Ansible discovers and configures from there. Each tool owns its layer cleanly.

2.2 Directory layout, file by file

```
# control-repo/ — the Git repo the control node clones and runs
ansible.cfg             # global settings: inventory path, forks, become, logging
requirements.yml        # Galaxy collections (amazon.aws, ansible.windows, …) — pinned
site.yml                # the estate push: one play for Linux, one for Windows
bootstrap.yml           # control node self-config: collections + SSH key from secret
local.yml               # ansible-pull shim (imports bootstrap.yml)
inventory/
  aws_ec2.yml           # DYNAMIC inventory — queries EC2, builds groups from tags
group_vars/
  all.yml               # region + secret name; per-field secret pattern (no bundle var)
  os_linux.yml          # Linux connection: ssh, become, fallback login user
  os_windows.yml        # Windows connection: winrm/5986, creds via inline lookup
  distro_amazon.yml     # login user ec2-user  (overrides os_linux)
  distro_ubuntu.yml     # login user ubuntu
roles/
  baseline/             # cross-OS baseline: timezone, chrony, base packages
    tasks/{main,linux,windows}.yml   defaults/main.yml   meta/main.yml
playbooks/
  ubuntu-setup.yml  ubuntu-apache2.yml  ubuntu-mysql.yml
  amazonlinux-setup.yml  amazonlinux-httpd.yml  amazonlinux-mysql.yml
  windows-adds.yml  windows-domain-join.yml  windows-iis.yml
  windows-python.yml  windows-share.yml  windows-zms-enforcer.yml
scripts/
  reconverge.sh         # git pull + refresh collections + run bootstrap.yml
  notify-result.sh      # log each run's result; SNS alert on failure
systemd/
  ansible-bootstrap.{service,timer}   # self-converge every 30 min
  ansible-estate.{service,timer}      # push site.yml every 60 min
  estate.env.example
.gitattributes .gitignore .ansible-lint .yamllint .pre-commit-config.yaml
```

Two "repos" that both say *repo*

`control_repo_url` is the *Git* repo the control node clones (this codebase, on GitHub). It is **not** a package mirror — OS packages install from the distro's built-in apt/dnf repos and Chocolatey's public community feed over the instances' NAT egress. No internal mirror is required.

2.3 The control node lifecycle & reconverge loop

Nobody SSHes in to "run Ansible". Terraform's cloud-init installs Ansible, clones this repo, and enables two **systemd timers**. From then on the node re-applies itself and the estate on a schedule, self-healing after reboots.

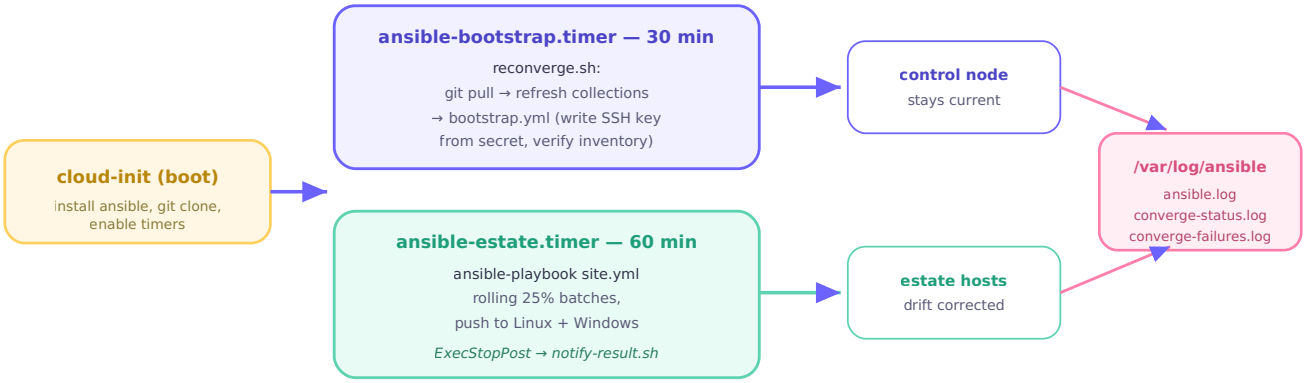

Figure 4 — Two timers. The 30-minute bootstrap keeps the control node itself current; the 60-minute estate timer pushes `site.yml`*. Every run's result lands in* `/var/log/ansible`*.*

This is your mental model for Part VII

When something "isn't applying," you're really asking one of: *did the timer fire?* (`systemctl list-timers`), *did the run succeed?* (`converge-status.log`), *what failed?* (`journalctl` / `ansible.log`). We'll walk each.

PART III

Inventory & Grouping

How the host list builds itself from EC2 tags, and how a host's tags decide both which plays touch it and how Ansible logs in.

3.1 Dynamic EC2 inventory

There is no hand-maintained list of servers. `inventory/aws_ec2.yml` uses the `amazon.aws.aws_ec2` plugin to query EC2 every run and build the inventory live. New instances appear automatically; terminated ones vanish.

```
plugin: amazon.aws.aws_ec2
regions:
  - "{{ lookup('ansible.builtin.env', 'AWS_REGION') | default('eu-west-3', true) }}"
filters:
  instance-state-name: running
  tag:ManagedBy: Terraform          # only instances Terraform built
hostnames:
  - private-ip-address              # control node is inside the VPC
compose:
  ansible_host: private_ip_address
keyed_groups:
  - { key: tags.OS,          prefix: os }       # → os_linux / os_windows
  - { key: tags.Distro,      prefix: distro }   # → distro_amazon / distro_ubuntu
  - { key: tags.Role,        prefix: role }     # → role_web / role_db / role_dc …
  - { key: tags.Environment, prefix: env }      # → env_zms …
```

Two filters do the gatekeeping

A host only appears if it is `running` *and* tagged `ManagedBy=Terraform` (capital T). An empty inventory almost always means the wrong `AWS_REGION` or a tag mismatch — not a broken plugin.

3.2 Tags become groups

Each `keyed_groups` rule turns a tag into a group named `prefix_value`. One instance therefore lands in several groups at once — and **group membership is how plays and variables find it**.

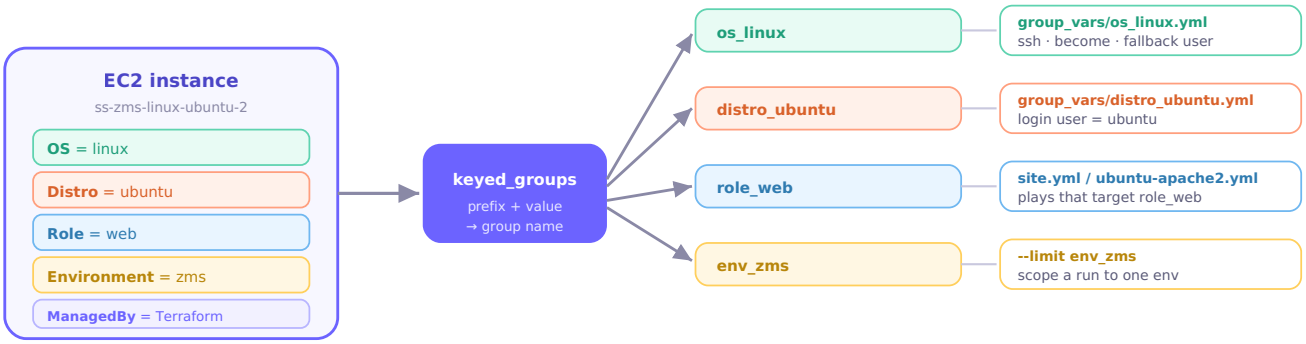

Figure 5 — One instance, four tags, four groups. Each group is a handle: connection `group_vars` attach to `os_*` / `distro_*`; plays target `role_*`; `--limit` can scope to `env_*`.

3.3 group_vars & the connection story

A host's groups decide *how Ansible logs in*. The rules live in `group_vars` and layer by precedence.

Linux: SSH, with the right user per distro

`group_vars/os_linux.yml` sets SSH + `sudo` and a **fallback** login user. But Ubuntu logs in as `ubuntu` and Amazon Linux as `ec2-user` — a real difference. The `distro_*` groups carry the correct user and, thanks to `ansible_group_priority: 10`, win over the `os_linux` default (otherwise alphabetical merge order would let `os_linux` clobber it).

```
# group_vars/distro_amazon.yml
ansible_group_priority: 10        # beat os_linux's default
linux_login_user: ec2-user      # distro_ubuntu.yml → ubuntu
```

Windows: WinRM over HTTPS

`group_vars/os_windows.yml` connects over WinRM on `5986`. Terraform stands up a self-signed listener, so we connect by private IP with `cert_validation: ignore`. Credentials are pulled *inline, field-scoped* from the secret (see Part IV) — no bundle variable.

```
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_transport: ntlm
ansible_winrm_server_cert_validation: ignore
ansible_user:     "{{ (lookup('amazon.aws.aws_secret', ansible_secret_name,
region=aws_region) | from_json).winrm_username }}"
ansible_password: "{{ (lookup('amazon.aws.aws_secret', ansible_secret_name,
region=aws_region) | from_json).winrm_password }}"
```

Estate	Connection	Port	Login user comes from	Auth
Ubuntu	`ssh`	22	`distro_ubuntu` → `ubuntu`	SSH key from secret

Amazon Linux	`ssh`	22	`distro_amazon` → `ec2-user`	SSH key from secret
Windows	`winrm`	5986	secret `winrm_username`	password from secret (TLS)

PART IV

Secrets

Where credentials live, how the control node learns which secret is its own, and why each playbook reads only the one field it needs.

4.1 The consolidated secret

Every deployment has **one** AWS Secrets Manager secret holding a single JSON document. Terraform creates and populates it from your `terraform.tfvars`; nothing sensitive is ever committed to Git.

```
# the one secret's JSON value (keys used across the playbooks)
{
  "ssh_private_key":      "-----BEGIN OPENSSH PRIVATE KEY-----\n…",
  "winrm_username":       "zmsadmin",
  "winrm_password":       "…",
  "provision_key":        "4|prod.zpath.net|…|1",   # ZMS enforcer nonce
  "dsrm_password":        "…",                       # AD safe-mode
  "domain_join_username": "ALCOR\\joinadmin",
  "domain_join_password": "…",
  "mysql_root_password":  "…"
}
```

4.2 Field-scoped resolution at point of use

The control node isn't told the secret's contents — it's told the secret's **name**, once, at build time. Terraform's cloud-init writes `ANSIBLE_SECRET_NAME` into `/etc/ansible/estate.env`; the systemd timers load that env; `group_vars/all.yml` reads it into `ansible_secret_name`. Each playbook then resolves **only the field it needs**, inline, at the moment it needs it.

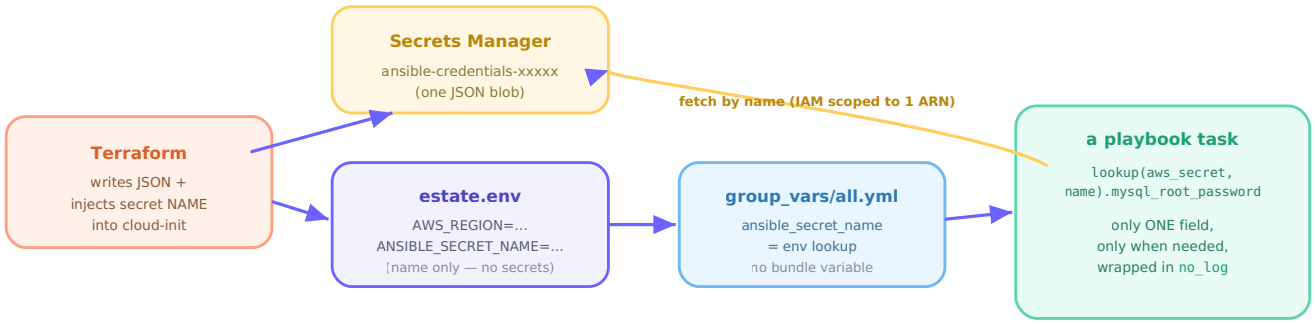

Figure 6 — Build time hands over a name; run time fetches the value. Removing the old global `ansible_credentials` bundle means no single variable ever holds the whole secret.

Why field-scoped?

A single broadly-scoped dict would put the SSH key and every password in scope for every play — one stray `debug` or `-vvv` could leak all of it. Reading one field at point of use, under `no_log`, keeps the blast radius tiny. The control node's IAM policy is scoped to that one secret ARN, so it can read nothing else.

PART V

Playbooks: Reading & Writing

The heart of the course. Learn the shape of a play, walk through the repo's real playbooks line by line, then write your own with confidence.

5.1 Anatomy of a play

A **playbook** is a YAML list of **plays**. A play binds a set of hosts to an ordered list of tasks (and/or roles). Every keyword below shows up in this repo — learn them once here.

Figure 7 — A play, annotated. `hosts` chooses the group; `tasks` run top-to-bottom; each task is one idempotent module call, optionally guarded by `when` and labelled with `tags`.

Keyword	Purpose
`hosts`	Which inventory group/pattern the play targets. Ours default to a group and accept `-e target=…`.
`gather_facts`	Whether to collect host facts first (needed for `ansible_os_family` checks).
`become`	Escalate privileges (`sudo`). On Windows, tasks run as the WinRM user.
`serial` / `max_fail_percentage`	Blast-radius controls: how many hosts at once, and when to abort.
`vars`	Variables scoped to the play.
`roles`	Reusable bundles applied before `tasks`.
`tasks`	The ordered work. Each item = one module call.
`when` / `tags` / `register` / `notify`	Per-task: condition, selector label, capture output, trigger a handler.

5.2 Walkthrough: `site.yml`

`site.yml` is what the estate timer runs. It has a pre-flight play plus one play per OS. Notice it targets the `os_linux` / `os_windows` groups and leans on `serial` for safe rollouts.

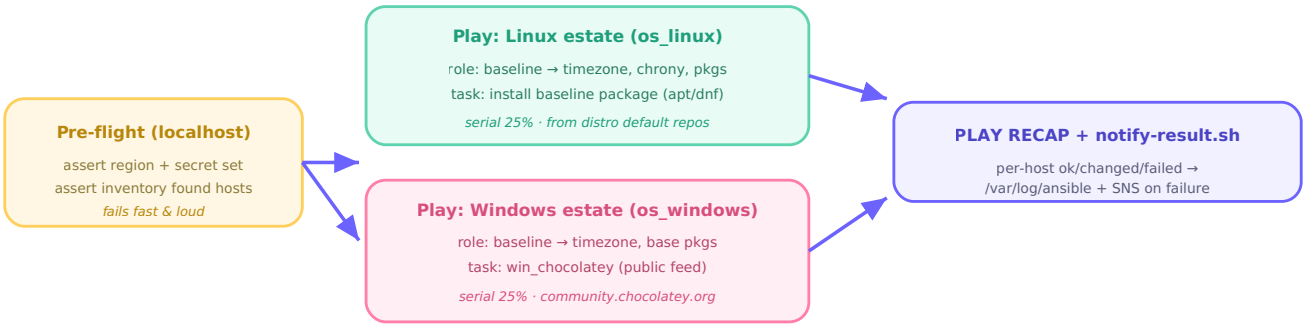

Figure 8 — `site.yml` flow: assert prerequisites, converge Linux, converge Windows (25% at a time), then record the result. If pre-flight fails, no host is touched.

```yaml
# site.yml — the Linux play (abridged)
- name: Linux estate
  hosts: os_linux
  serial: "{{ rolling_batch | default('100%') }}"
  roles:
    - { role: baseline, tags: [baseline] }
  tasks:
    # Packages install from the distro's built-in apt/dnf repos — no internal mirror.
    - name: Install baseline package
      ansible.builtin.package:
        name: "{{ linux_baseline_package | default('htop') }}"
        state: present
      tags: [packages]
```

Try it safely

`ansible-playbook site.yml --check --diff` runs the whole thing in *dry-run*: it reports what *would* change without touching a single host. This is the safest way to see estate drift.

5.3 Walkthrough: a Linux playbook

`playbooks/ubuntu-mysql.yml` installs MySQL on Ubuntu DB hosts. It shows three patterns you'll reuse: a **safety guard**, a **field-scoped secret**, and an **idempotent** privileged change.

```yaml
- name: Install and enable MySQL
  hosts: "{{ target | default('distro_ubuntu') }}"   # ① sane default group
  vars:
    # ② read ONLY this one field, inline, at point of use
    mysql_root_password: "{{ (lookup('amazon.aws.aws_secret', ansible_secret_name,
region=aws_region) | from_json).mysql_root_password | default('', true) }}"
  tasks:
    - name: Skip non-Debian hosts
      ansible.builtin.meta: end_host              # ③ guard: bail on Amazon Linux
      when: ansible_os_family != "Debian"

    - name: Install MySQL server and the Python client
      ansible.builtin.apt:
        name: [mysql-server, python3-pymysql]
        state: present

    - name: Set the MySQL root password (switch root to password auth)
      community.mysql.mysql_user:
        name: root
        password: "{{ mysql_root_password }}"
        plugin: mysql_native_password           # ④ so the password actually works
        check_implicit_admin: true              #    idempotent across reruns
        login_unix_socket: /run/mysqld/mysqld.sock
        state: present
      no_log: true                                # ⑤ never print the secret
      when: mysql_root_password | length > 0
```

	Pattern	Why it's here
①	Default to a distro group	Run with no args and it targets just Ubuntu; override with `-e target=…` or `--limit`.
②	Field-scoped secret	Only `mysql_root_password` is fetched, only in this play (Part IV).
③	`meta: end_host` guard	Belt-and-suspenders — if the play is aimed wider, non-Debian hosts drop out cleanly.
④	`mysql_native_password`	Fresh Ubuntu root uses `auth_socket` (password ignored); this switches it so auth works.
⑤	`no_log: true`	Redacts the task so the password never lands in logs or `-v` output.

5.4 Walkthrough: a Windows playbook

`playbooks/windows-adds.yml` promotes the first Windows host into a new Active Directory forest. Windows work uses the `ansible.windows` / `microsoft.ad` collections and often needs reboots — Ansible handles those inline.

```yaml
- name: Build new AD forest (alcor.co.in)
  hosts: "{{ target | default('role_dc') }}"      # the role_dc group (Domain_Controller)
  vars:
    dsrm_password: "{{ (lookup('amazon.aws.aws_secret', ansible_secret_name,
region=aws_region) | from_json).dsrm_password }}"
  tasks:
    - name: Assert the DSRM password is present
      ansible.builtin.assert:                      # fail early with a clear message
        that: [ "dsrm_password | length > 0" ]
      no_log: true

    - name: Install AD DS role
      ansible.windows.win_feature:
        name: AD-Domain-Services
        include_management_tools: true
      register: ad_feature

    - name: Reboot if required
      ansible.windows.win_reboot:
      when: ad_feature.reboot_required             # only when the module says so

    - name: Promote to first DC of the new forest
      microsoft.ad.domain:
        dns_domain_name: alcor.co.in
        safe_mode_password: "{{ dsrm_password }}"
        install_dns: true
        reboot: true                             # module manages the post-promo reboot
```

Windows idioms to notice

`register` captures a task's result so a later `when` can react to it (reboot only if needed). `assert` turns a missing prerequisite into a clean, early failure instead of a confusing one three tasks later. `until` / `retries` (used further down for the ADWS service) polls until something comes up.

5.5 Roles: the `baseline` role

A **role** packages related tasks, variables, defaults, and handlers into a reusable directory you can drop into any play with one line. `site.yml` applies `baseline` to every host. Its structure is the standard Ansible layout:

```
roles/baseline/
  tasks/main.yml       # entry point — dispatches by OS
  tasks/linux.yml      # timezone, base packages, chrony
  tasks/windows.yml    # timezone, base choco packages
  defaults/main.yml    # overridable defaults (timezone, package lists)
  meta/main.yml        # role metadata / supported platforms
```

```yaml
# roles/baseline/tasks/main.yml — one entry point, two OS paths
- name: Include Linux baseline tasks
  ansible.builtin.include_tasks: linux.yml
  when: ansible_os_family != "Windows"

- name: Include Windows baseline tasks
  ansible.builtin.include_tasks: windows.yml
  when: ansible_os_family == "Windows"
```

Defaults live in `defaults/main.yml` (the *lowest* precedence, so any group_var or `-e` overrides them). That's why the baseline timezone or package list can be changed per environment without editing the role.

5.6 How to write your own playbook

Say you want to install and run **nginx** on the Ubuntu web hosts. Follow the same six steps every repo playbook uses.

1. **Pick the target group.** Web hosts are `role_web`; Ubuntu-only means `distro_ubuntu`. Default to one, allow an override.
2. **Decide privilege & facts.** Installing packages needs `become: true`; the `when` guard needs `gather_facts: true`.
3. **Write idempotent tasks** with fully-qualified module names (`ansible.builtin.apt`, not `apt`).
4. **Guard & tag.** Add a `meta: end_host` for the wrong OS and `tags` for selective runs.
5. **Use a handler** for "restart on change" so nginx only reloads when its config actually changed.
6. **Test before you push:** `--syntax-check`, then `--check --diff` against one host with `--limit`.

```yaml
# playbooks/ubuntu-nginx.yml — a complete, idiomatic example
- name: Install and enable nginx
  hosts: "{{ target | default('role_web') }}"
  gather_facts: true
  become: true
  tasks:
    - name: Skip non-Debian hosts
      ansible.builtin.meta: end_host
      when: ansible_os_family != "Debian"

    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600
      tags: [nginx]

    - name: Deploy the site config
      ansible.builtin.copy:
        src: files/site.conf
        dest: /etc/nginx/sites-available/default
        mode: "0644"
      notify: Reload nginx            # fire the handler only if this changed
      tags: [nginx]

    - name: Ensure nginx is running and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
      tags: [nginx]

  handlers:
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

The idempotency test

Run your playbook twice. The **second** run should report `changed=0`. If it keeps reporting `changed`, a task isn't declaring desired state properly (a common culprit: `command` / `shell` without `creates:` or `changed_when:`). Prefer a real module over `shell` whenever one exists.

Pre-commit gates

The repo ships `.ansible-lint`, `.yamllint`, and `.pre-commit-config.yaml`. Run `pre-commit run --all-files` before pushing — it catches unqualified modules, bad indentation, and missing `changed_when` for you.

PART VI

Running & Targeting

The commands you'll type by hand between automated converges, and how to aim them precisely.

6.1 The commands you'll use daily

Always load the environment first

The systemd timers get `AWS_REGION` and `ANSIBLE_SECRET_NAME` from `estate.env`, but your interactive shell does not. Start every manual session with:

```
cd /opt/control-repo
set -a; source /etc/ansible/estate.env; set +a
```

```
# Converge the whole estate
ansible-playbook site.yml

# Dry run — report drift, change nothing (safest first move)
ansible-playbook site.yml --check --diff

# Run one playbook against its default group
ansible-playbook playbooks/ubuntu-apache2.yml

# Ad-hoc: run a single module without a playbook
ansible os_linux -m ansible.builtin.ping
ansible role_web -m ansible.builtin.command -a "systemctl status nginx"

# Validate before you push
ansible-playbook playbooks/ubuntu-nginx.yml --syntax-check
pre-commit run --all-files
```

6.2 Targeting: limits, tags, check mode, batches

Flag	Effect
`--limit web01`	Restrict the run to one host (or a group / pattern like `'role_web:&env_zms'`).
`--tags packages`	Run only tasks carrying that tag; `--skip-tags` excludes.
`--check`	Dry run — predict changes, apply none.
`--diff`	Show the before/after of changed files/templates.
`-e "target=role_db"`	Override the playbook's default host group at run time.
`-e rolling_batch=25%`	Converge 25% of hosts at a time (blast-radius control).
`--start-at-task "Name"`	Resume a run from a specific task after a fix.
`-vvv`	

Verbose — show connection details and module arguments (secrets stay redacted by `no_log`).

Combine them

`ansible-playbook site.yml --limit os_windows --tags baseline --check --diff` = "show me, without changing anything, what the baseline role *would* do to just the Windows hosts."

PART VII

Troubleshooting & Status

A field manual. Four questions — is the inventory healthy, can we reach the hosts, did the converge succeed, and what failed — plus a decision tree and a failure-mode table.

7.1 Is the inventory healthy?

```
# Full tree of discovered groups/hosts (os_linux, distro_amazon, role_web, …)
ansible-inventory -i inventory/aws_ec2.yml --graph

# Everything as YAML, with host vars (ansible_host, tags…)
ansible-inventory -i inventory/aws_ec2.yml --list

# Confirm a specific group is populated
ansible-inventory -i inventory/aws_ec2.yml --graph os_windows
```

Empty graph?

It's almost always one of two things: `echo $AWS_REGION` doesn't match where the instances actually run, or the instances aren't tagged `ManagedBy=Terraform`. It is *not* a broken plugin.

7.2 Can we reach the hosts?

```
# Linux: SSH + Python round-trip
ansible os_linux   -i inventory/aws_ec2.yml -m ansible.builtin.ping

# Windows: WinRM round-trip
ansible os_windows -i inventory/aws_ec2.yml -m ansible.windows.win_ping

# One host, verbose, to see the exact connection attempt
ansible web01 -i inventory/aws_ec2.yml -m ping -vvv
```

Symptom	Likely cause
Linux `UNREACHABLE` / permission denied (publickey)	Wrong login user (Amazon Linux got `ubuntu`?) or the SSH key isn't readable by the run user — check `distro_*` groups and `/etc/ansible/keys`.
Windows `ping` works but `win_ping` fails	WinRM listener/security-group issue on 5986, or credentials — not a network route problem.
All hosts unreachable	Security groups, or the control node lost its SSH key (run `bootstrap.yml`).

7.3 Did the converge succeed?

```
# The timers and their next/last fire
systemctl list-timers 'ansible-*'
systemctl status ansible-estate.service ansible-bootstrap.service

# Exit status + result of the LAST run of each unit
systemctl show ansible-estate.service -p ExecMainStatus -p Result -p ActiveEnterTimestamp

# Per-run result log (written by scripts/notify-result.sh)
tail -n 20 /var/log/ansible/converge-status.log

# Dry-run drift check — changed=0 means converged
ansible-playbook site.yml --check --diff
```

7.4 Reading the errors

```
# Failures-only log (notify-result.sh appends here on any failed run)
cat /var/log/ansible/converge-failures.log

# Full run log (log_path in ansible.cfg) — grep the failures
grep -nE 'fatal|FAILED|UNREACHABLE|failed=' /var/log/ansible/ansible.log | tail -n 40

# systemd journal for a unit — full stderr/traceback
journalctl -u ansible-estate.service -n 100 --no-pager

# Reproduce live, verbosely, narrowed to the failing host
ansible-playbook site.yml --limit web01 -vvv
ansible-playbook site.yml --start-at-task "Install baseline package"
```

7.5 Troubleshooting decision tree

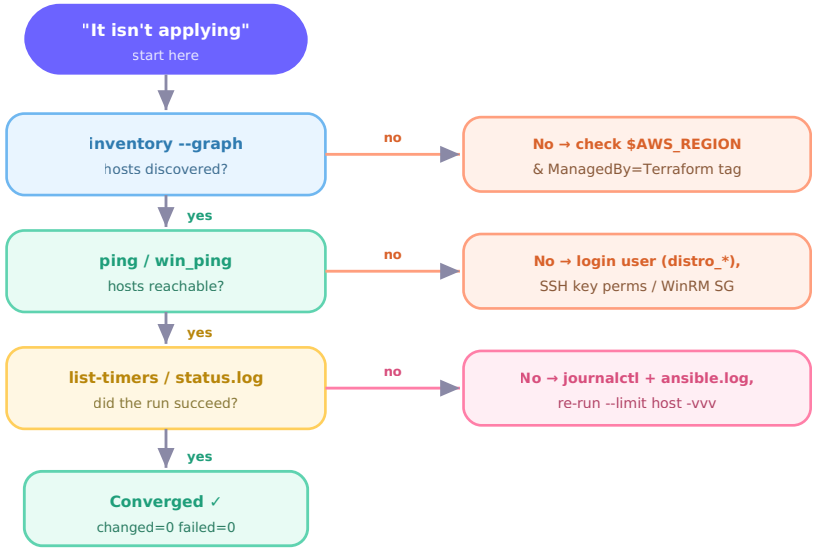

Figure 9 — Work top to bottom. Each "yes" moves you down a layer; each "no" points at the specific fix. Most incidents resolve at the inventory or reachability layer.

7.6 Common failure modes

Message / symptom	Root cause	Fix
Inventory graph empty	Wrong region or missing tag	Fix `AWS_REGION`; confirm `ManagedBy=Terraform`
`Permission denied (publickey)` on Amazon Linux	Logged in as `ubuntu`	Ensure the `distro_amazon` group + `ec2-user`
`UNREACHABLE` reading the key	Key not readable by run user	Key owned by the service user in `/etc/ansible/keys`; re-run `bootstrap.yml`
WinRM: cannot connect on 5986	Listener not up / SG closed	Check the instance's WinRM listener & the control-node SG rule
Secret lookup fails	`ANSIBLE_SECRET_NAME` unset	`source /etc/ansible/estate.env`; verify the name & IAM
Playbook keeps reporting `changed`	Non-idempotent task	Add `creates:` / `changed_when:` or use a real module
Timer never fired	Unit not enabled	`systemctl enable --now ansible-estate.timer`

APPENDICES

Quick References

Tear-out pages you'll return to: the command cheat sheet, a glossary, and every module this repo uses.

Appendix A — Command cheat sheet

Run everything from `/opt/control-repo` after `set -a; source /etc/ansible/estate.env; set +a`.

Goal	Command
See inventory	`ansible-inventory -i inventory/aws_ec2.yml --graph`
Inventory detail	`ansible-inventory -i inventory/aws_ec2.yml --list`
Ping Linux	`ansible os_linux -m ansible.builtin.ping`
Ping Windows	`ansible os_windows -m ansible.windows.win_ping`
Converge estate	`ansible-playbook site.yml`
Dry run + drift	`ansible-playbook site.yml --check --diff`
One host	`ansible-playbook site.yml --limit web01`
One tag	`ansible-playbook site.yml --tags packages`
Override target	`ansible-playbook playbooks/ubuntu-mysql.yml -e target=role_db`
Syntax check	`ansible-playbook playbooks/<name>.yml --syntax-check`
Timers	`systemctl list-timers 'ansible-*'`
Last run result	`tail -n 20 /var/log/ansible/converge-status.log`
Failures only	`cat /var/log/ansible/converge-failures.log`
Unit journal	`journalctl -u ansible-estate.service -n 100 --no-pager`
Lint before push	`pre-commit run --all-files`

Appendix B — Glossary

Term	Definition
Ad-hoc command	A one-off `ansible` module run without a playbook.
Become	Privilege escalation (`sudo`).
Check mode	`--check` : predict changes without applying them.
Collection	Versioned bundle of modules/plugins from Galaxy.
Converge	One full application of desired state to hosts.

Dynamic inventory	A host list generated at run time (here, from EC2).
Fact	Auto-discovered host data (`ansible_os_family` …).
FQCN	Fully-qualified collection name, e.g. `ansible.builtin.apt` .
Handler	Task that runs only when notified by a change.
Idempotent	Re-running changes nothing already correct.
Keyed group	Inventory group auto-built from a tag value.
Play / Playbook	Hosts-to-tasks mapping / file of plays.
Role	Reusable directory of tasks/vars/handlers.
Register	Capture a task's result into a variable.
Serial	How many hosts converge at once.

Appendix C — Modules used in this repo

Module (FQCN)	Does	Seen in
`amazon.aws.aws_ec2`	Dynamic inventory plugin	`inventory/aws_ec2.yml`
`amazon.aws.aws_secret`	Read a Secrets Manager value (lookup)	`group_vars` , playbooks
`ansible.builtin.apt`	Debian/Ubuntu packages	ubuntu-*
`ansible.builtin.dnf`	RHEL/Amazon Linux packages	amazonlinux-*
`ansible.builtin.package`	OS-agnostic package install	`site.yml` , baseline
`ansible.builtin.service`	Manage a service	apache2/httpd/mysql
`ansible.builtin.copy`	Write file content (SSH key, configs)	`bootstrap.yml`
`ansible.builtin.assert`	Fail early on unmet prerequisites	windows-adds, site.yml
`ansible.builtin.meta: end_host`	Cleanly drop the wrong-OS host	most playbooks
`community.mysql.mysql_user`	Manage MySQL/MariaDB users	*-mysql
`ansible.windows.win_feature`	Install Windows roles/features	windows-adds, iis
`ansible.windows.win_reboot`	Reboot and wait	windows-adds, iis
`ansible.windows.win_package`	Install an MSI	windows-zms-enforcer
`microsoft.ad.domain / .membership`	Promote / join AD	windows-adds, -domain-join
`chocolatey.chocolatey.win_chocolatey`	Windows packages (public feed)	`site.yml` , windows-python

End of course

You now have the concepts, the repo's real examples, and the operational commands to run, target, and troubleshoot a converge. Keep Parts VI–VII and the appendices within reach — they're the day-to-day. Everything here mirrors the repository as it currently stands.